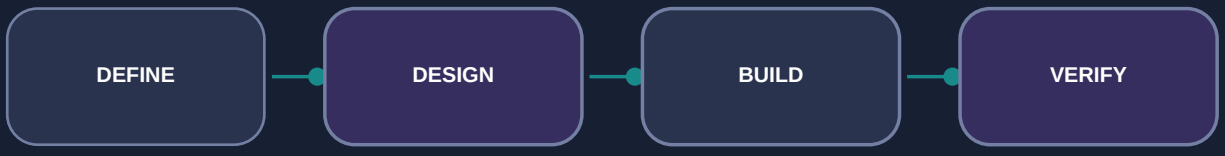
DOCUMENT DB • Backend Schema Document

Backend Schema Document

Entity relationships, schema reference and integrity controls



Project name: Slotwise (provisional)

Proposed stack: React + Vite + Tailwind • Node.js + Express • Vercel

Public-source clean-room plan • illustrative graphics, not product screenshots

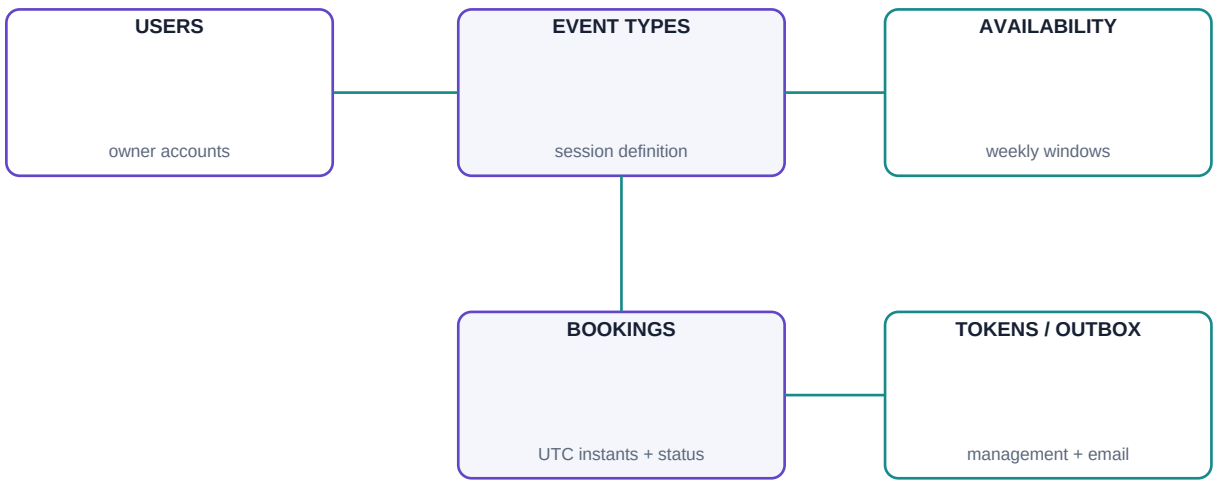
Visual summary

Backend Schema Document

PURPOSE

Show the seven-table reference model and the integrity rules that protect bookings.

KEY VISUAL



Conceptual relationship diagram; full DDL remains in the source content.

REVIEW NOTE

Original content is retained after this summary; proposed visuals are illustrative, not real product screenshots.

DOCUMENT 05 OF 6

Backend Schema Document

Data model, PostgreSQL DDL, API examples and integrity controls

Project: Slotwise — provisional independent scheduling product

Reference: Calendly public website and help documentation; no private access.

Platform: Web | **Proposed frontend:** React + Vite + Tailwind CSS | **Backend:** Node.js + Express

Hosting preference: Vercel | **Database:** unresolved conflict between MongoDB preference and required PostgreSQL DDL

Inputs used / assumptions

The user's preferred database is MongoDB, while the requested specification explicitly requires PostgreSQL DDL and database-level booking constraints. This document provides a complete PostgreSQL reference schema; approval is required before implementation. The schema is for the proposed Slotwise core loop, not for Calendly's internal database.

Evidence discipline: Verified public capabilities are distinguished from design proposals and guesses. No review corpus, actual application screenshots, private account data, or production credentials were provided.

1. Data architecture and entity relationships

The reference schema has seven core tables: users, sessions, event_types, availability_rules, bookings, management_tokens and outbox_jobs. All IDs are UUIDs. Instants use TIMESTAMPTZ in UTC; the IANA time-zone name is stored separately.

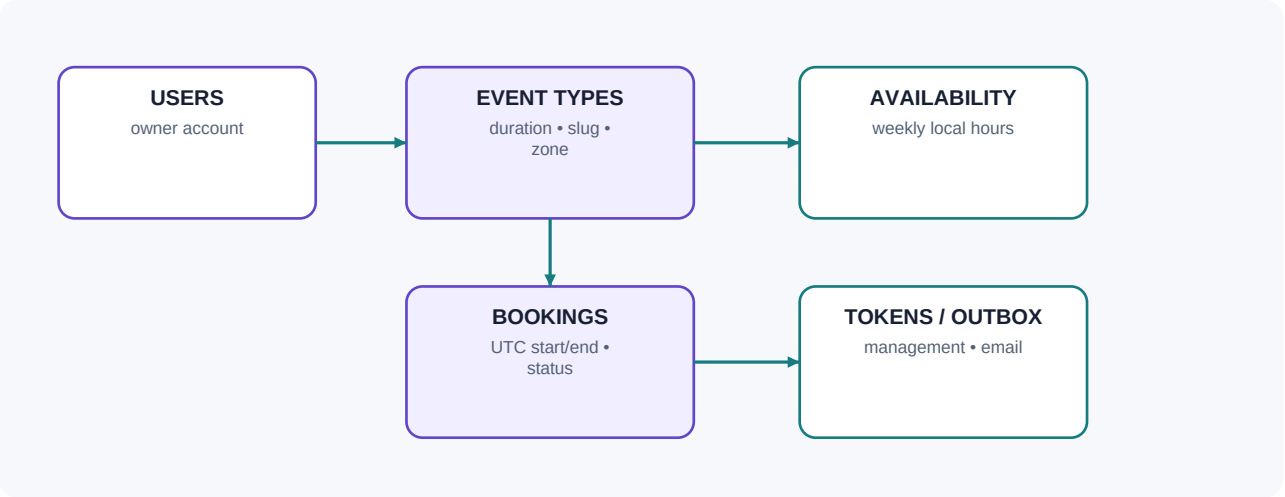


Figure 1. Simplified ERD. Relationships are shown conceptually; see DDL for explicit constraints and delete rules.

2. Entity inventory

Entity	Purpose	Owner / relationships	Confidence
users	Owner accounts	Root entity	High
sessions	Authenticated sessions	user_id → users; cascade on user deletion	High
event_types	Published or draft session definitions	user_id → users; unique slug	High
availability_rules	Weekly local availability windows	event_type_id → event_types	High
bookings	Confirmed/canceled appointment records	user_id, event_type_id; overlap constraint	High
management_tokens	Hashed cancellation/reschedule tokens	booking_id → bookings	High
outbox_jobs	Durable notification jobs	booking_id nullable; retry state	High

3. PostgreSQL DDL

The following SQL is a reference migration. It requires PostgreSQL with pgcrypto and btree_gist support. Apply migrations in a controlled deployment step and validate them against the selected hosting provider.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE users (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
email TEXT NOT NULL UNIQUE,
password_hash TEXT NOT NULL,
role TEXT NOT NULL DEFAULT 'owner'
CHECK (role IN ('owner','admin')),
email_verified_at TIMESTAMPTZ,
deleted_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE sessions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
token_hash TEXT NOT NULL UNIQUE,
expires_at TIMESTAMPTZ NOT NULL,
revoked_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX sessions_user_id_idx ON sessions(user_id);
CREATE INDEX sessions_expires_at_idx ON sessions(expires_at);

CREATE TABLE event_types (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
title TEXT NOT NULL CHECK (length(title) BETWEEN 1 AND 120),
slug TEXT NOT NULL UNIQUE,
description TEXT,
duration_minutes INTEGER NOT NULL CHECK (duration_minutes BETWEEN 5 AND 480),
location_type TEXT NOT NULL DEFAULT 'custom'
CHECK (location_type IN ('custom','phone','online')),
location_details TEXT,
timezone TEXT NOT NULL,
is_published BOOLEAN NOT NULL DEFAULT false,
archived_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX event_types_user_id_idx ON event_types(user_id);
CREATE INDEX event_types_owner_status_idx
ON event_types(user_id,is_published,archived_at);

CREATE TABLE availability_rules (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
event_type_id UUID NOT NULL REFERENCES event_types(id) ON DELETE CASCADE,
weekday INTEGER NOT NULL CHECK (weekday BETWEEN 0 AND 6),
start_local TIME NOT NULL,
end_local TIME NOT NULL,
timezone TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CHECK (start_local < end_local)
);
CREATE INDEX availability_event_type_idx ON availability_rules(event_type_id);
CREATE INDEX availability_weekday_idx ON availability_rules(event_type_id,weekday);

CREATE TABLE bookings (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
user_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
event_type_id UUID NOT NULL REFERENCES event_types(id) ON DELETE RESTRICT,
invitee_name TEXT NOT NULL CHECK (length(invitee_name) BETWEEN 1 AND 120),
invitee_email TEXT NOT NULL,
invitee_note TEXT,
start_at TIMESTAMPTZ NOT NULL,
end_at TIMESTAMPTZ NOT NULL,
timezone TEXT NOT NULL,
status TEXT NOT NULL DEFAULT 'confirmed'
CHECK (status IN ('confirmed','canceled','completed','no_show')),
idempotency_key TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CHECK (start_at < end_at),
UNIQUE (user_id,idempotency_key)
);
CREATE INDEX bookings_user_id_idx ON bookings(user_id);
CREATE INDEX bookings_event_type_idx ON bookings(event_type_id);
CREATE INDEX bookings_start_at_idx ON bookings(start_at);
CREATE INDEX bookings_status_idx ON bookings(status);
CREATE INDEX bookings_owner_start_idx ON bookings(user_id,start_at DESC);

ALTER TABLE bookings ADD CONSTRAINT bookings_no_host_overlap
EXCLUDE USING gist (
user_id WITH =,
tstzrange(start_at,end_at,'[)') WITH &&
)
WHERE (status = 'confirmed');

CREATE TABLE management_tokens (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
token_hash TEXT NOT NULL UNIQUE,
expires_at TIMESTAMPTZ NOT NULL,
used_at TIMESTAMPTZ,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX management_tokens_booking_idx ON management_tokens(booking_id);
CREATE INDEX management_tokens_expires_idx ON management_tokens(expires_at);

CREATE TABLE outbox_jobs (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
booking_id UUID REFERENCES bookings(id) ON DELETE SET NULL,
job_type TEXT NOT NULL CHECK (job_type IN
('booking_confirmation','booking_canceled','booking_rescheduled')),
status TEXT NOT NULL DEFAULT 'pending' CHECK (status IN
('pending','processing','sent','failed')),
payload JSONB NOT NULL DEFAULT '{}'::jsonb,
attempts INTEGER NOT NULL DEFAULT 0 CHECK (attempts >= 0),
available_at TIMESTAMPTZ NOT NULL DEFAULT now(),
processed_at TIMESTAMPTZ,
last_error_code TEXT,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX outbox_status_available_idx ON outbox_jobs(status,available_at);
CREATE INDEX outbox_booking_idx ON outbox_jobs(booking_id);
```

4. Integrity and implementation notes

- The exclusion constraint prevents overlapping confirmed appointments for one host, even across different event types.
- Use a transaction to validate the slot, insert the booking and insert an outbox job. Roll back all writes if the reservation fails.
- Do not send email before commit; a worker processes the outbox after commit.
- Maintain updated_at in application code or with a database trigger.
- PostgreSQL TIMESTAMPTZ stores an instant, not the original IANA zone; store timezone separately.
- Unique(user_id, idempotency_key) protects non-null keys; application logic must deliberately handle absent keys and retries.
- ON DELETE RESTRICT for bookings preserves referenced history. Account deletion requires an explicit retention/anonymization path.

5. Access rules

Entity	Policy
users	User may access own profile; privileged administration is separate
sessions	Only owner can inspect/revoke; server validates session hash
event_types	Owner-only CRUD scoped by authenticated user ID
availability_rules	Accessible through an event owned by current user
bookings	Owner sees own bookings; public visitor can create a valid booking
management_tokens	Store only token hash; no raw token in database
outbox_jobs	Worker-only; no direct user access

This baseline uses data-layer authorization, not Row Level Security. Every private query must include the authenticated owner's ID. If RLS is added later, define and test policies for every table and ensure the application role cannot bypass them.

6. API and representative JSON

Endpoint	Input	Success output
POST /api/v1/public/:slug/bookings	event slug, slot start, invitee name/email, idempotency key	201 + booking reference and confirmed start/end
GET /api/v1/public/:slug/slots	date range and display zone	Available slots with UTC instants and labels
GET /api/v1/bookings	date/status filters; owner session	Own booking list only
POST /api/v1/manage/:token/cancel	management token	204 or updated canceled booking

```
{ "eventSlug": "intro-tutoring", "startAt": "2026-11-02T04:30:00Z", "timezone": "Asia/Kolkata", "invitee": { "name":
"Alex Example", "email": "alex@example.test" }, "idempotencyKey": "demo-request-001" }
```

The JSON above is synthetic example data, not a real customer record. The API must derive the owner and event configuration from the validated public slug, not from a client-supplied owner ID.

7. Error response format

```
{ "error": { "code": "SLOT_UNAVAILABLE", "message": "That time is no longer available. Choose another slot.",
"requestId": "req_example", "fieldErrors": {} } }
```

8. Indexes, migrations, backups and deletion

- Foreign-key columns are indexed; owner, status, date and expiration filters have dedicated indexes.
- Use versioned migrations and review them before deployment.

- Automate daily backups and test restore procedures.
- Define RPO/RTO appropriate to the service before launch.
- GDPR deletion path: verify requester, revoke sessions, handle future bookings, anonymize/delete eligible personal data, revoke management tokens, minimize log/job data, retain only records required by law, and record completion without excess personal data.

9. MongoDB alternative and open decision

If MongoDB is retained, this PostgreSQL schema must be translated into collections and indexes. The SQL exclusion constraint cannot be copied directly. A MongoDB implementation must explicitly design and test atomic slot reservation, idempotency, and cross-event host overlap protection before release. Do not run both databases for the MVP.